

MajorTom

A major-version upgrade is one of the most feared tasks in software maintenance.

Not because of the diff.

Because the diff looks right and the software is still broken.

The problem

- `app.del()` removed
- `res.send(status)` removed
- `/*` must be named
- `/:format?` -> braces
- regex paths rejected

A regex produces a codebase that looks migrated
and fails at runtime.

The solution

The vendor's own migration guide is the source of truth.
Apply only what that guide justifies. Then prove it by
running the tests.

```
$ npm install && npm run demo
```

No API keys. No setup. No network required.

The pipeline

1. INTAKE

2. PLAN

3. IMPACT

4. BASELINE

5. EXECUTE

6. VERIFY

7. REPORT

Every stage is checkpointed and resumable.

Invariants

I1 never touches main; no force-push, no merge

I2 every edit cited; coverage computed from records

I3 the guide is DATA; injected instructions ignored

Honest outcomes

reports NOT GREEN when the migration is not green

Proof

express 4.21.2 -> 5.1.0

GREEN - 9 files changed

citation coverage 100%

tests 15/16 -> 15/16

exit code 0

Honest accounting

A pre-existing failure is recorded in the baseline and excluded. A regression is reported as one.

1 pre-existing failure: excluded, not blamed

attributable to migration: NONE

A failing run and a passing run are equally easy to produce.

Run it yourself

```
$ npm install
```

```
$ npm run demo
```

MIT licensed. 138 tests. 7 stages. 100% cited.